Backend Application → MVC Architecture

app.js

```
let express = require('express');
let app = express();
let cors = require('cors');
const morgan = require('morgan');


//middleware
app.use(cors());
app.use(morgan("dev"));
app.use(express.json());
app.use(express.urlencoded({extended: true}));


//routes



let PORT = 8080;
app.listen(PORT,()⇒{
   console.log(`Server is running on port ${PORT}`);
})
```

model → users

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
```

```
  email: {
    type: String,
    required: true,
  },
  password: {
    type: String,
    required: true,
  },
  name: String,
  token: {
    type: String,
    required: true,
  },
  role: {
    type: String,
    default: "user",
  },
});

const User = mongoose.model("User", userSchema);

module.exports = { User };
```

DB folder → connectDB.js

```
let mongoose = require('mongoose');


function connectDB(){
      mongoose.connect('mongodb://localhost:27017/movieApp')
   .then(()⇒{
      console.log("db is connected")
   }).catch((err)⇒{
```

```
        console.log("db is not connected",err);
    })
}

module.exports = connectDB;
```

App.js after dbconnected

```
let express = require('express');
let app = express();
let cors = require('cors');
const morgan = require('morgan');

//calling db
connectDB();

//middleware
app.use(cors());
app.use(morgan("dev"));
app.use(express.json());
app.use(express.urlencoded({extended: true}));


//routes



let PORT = 8080;
app.listen(PORT,()=>{
    console.log(`Server is running on port ${PORT}`);
}
```

routes folder → user.js

```
const express = require("express");
const router = express.Router();
const {User} = require('../models/User');
const { signup, login } = require("../controllers/users.controller");


//task-1 → route for register
router.post('/register',signup);

//task 2 →route for login
router.post('/login',login );


module.exports = router;
```

controllers → user.controller.js

register

```
const {User} = require('../models/User');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');


const signup = async (req, res) => {
  try {
    const { email, password, name } = req.body;

    // Check if any field is missing
    if (!email || !password || !name) {
      return res.status(400).json({ message: "Some fields are missing" });
    }

    // Check if the user already exists
```

```
        const isUserAlreadyExist = await User.findOne({ email });

        if (isUserAlreadyExist) {
            res.status(400).json({ message: "User already has an account" });
            return;
        }else{
            // Hash the password
             const salt = bcrypt.genSaltSync(10);
            const hashedPassword = bcrypt.hashSync(password, salt);
            // Generate JWT token
            const token = jwt.sign({ email }, "supersecret", { expiresIn: "365d" });

            // Create user in database
            await User.create({
                name,
                email,
                password: hashedPassword,
                token,
                role:"user",
            });
            return res.status(201).json({ message: "User created successfully" });

        }
    } catch (error) {
        console.error(error);
        return res.status(500).json({ message: "Internal Server Error" });
    }
}
```

login

```
const login = async (req, res) => {
    try {
        const { email, password } = req.body;
```

```
        // Check if email or password is missing
        if (!email || !password) {
            return res.status(400).json({ message: "Email and Password are required"
        }

        // Find user by email
        const user = await User.findOne({ email });

        if (!user) {
            return res.status(400).json({ message: "User is not registered. Please reg
        }

        //  Compare the entered password with the stored hashed password
        const isPasswordMatched = bcrypt.compareSync(password, user.password

        if (!isPasswordMatched) {
            return res.status(400).json({ message: "Password not matched" });
        }

        //  Successful login - Return user data
        return res.status(200).json({
            id: user._id,
            name: user.name,
            token: user.token,
            email: user.email,
            role: user.role,
        });

    } catch (error) {
        console.error("Error during login:", error);
        return res.status(500).json({ message: "Internal Server Error" });
    }
}
```

```
module.exports = {signup,login}
```

index.js

```
const UserRoutes = require("./user");
const express = require("express");
const router = express.Router();

router.use("/user",UserRoutes);

module.exports = router;
```

app.js

```
let express = require('express');
let app = express();
let cors = require('cors');
const morgan = require('morgan');

//calling db
connectDB();

//middleware
app.use(cors());
app.use(morgan("dev"));
app.use(express.json());
app.use(express.urlencoded({extended: true}));


//routes
app.use(routes);
```

```
let PORT = 8080;
app.listen(PORT,()=>{
   console.log(`Server is running on port ${PORT}`);
}
```

Product

productSchema →

```
const mongoose = require("mongoose");

const productSchema = new mongoose.Schema({
  name: {
    type: String,
  },
  price: {
    type: Number,
  },
  brand: {
    type: String,
  },
  stock: {
    type: Number,
  },
  image: {
    type: String,
  },
  description: {
    type: String,
  },
  user: {
    type: mongoose.Schema.ObjectId,
    ref: "User",
```

```
  },
});

const Product = mongoose.model("Product", productSchema);

module.exports = { Product };
```

routes→ product.js

```
const express = require("express");
const router = express.Router();
const {products,addProduct,SingleProduct,productUpdate,deleteProduct} = requ

//task3 → get the product
router.get('/products', products)

//task4→ add a product
router.post('/add-product',addProduct)

//task5 → to show the particular product
router.get('/product/:id',SingleProduct)


//task-6 update product
router.patch("/product/edit/:id",productUpdate );

//task-7 → delete a product
router.delete("/product/delete/:id",deleteProduct);


module.exports = router;
```

controllers → product.controller.js

products

```
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const {Product} = require('../models/Product');


const products = async(req,res)⇒{
   try{
      const products = await Product.find();
      res.status(200).json({
         products:products
      })
   }catch (error) {
      return res.status(500).json({ message: "Internal Server Error" });
   }
}
```

addProduct

```
const addProduct = async(req,res)⇒{
   try{
      const body = req.body;
      const name = body.name;
      const { token } = req.headers;
      const decodedtoken = jwt.verify(token, "supersecret");
      // console.log("✌️decodedtoken ⟶", decodedtoken);
      const user = await User.findOne({ email: decodedtoken.email });
      const description = body.description;
      const image = body.image;
      const price = body.price;
      const brand = body.brand;
      const stock = body.stock;
```

```
        /// now we aassuming we hace every thing for product
        await Product.create({
          name: name,
          description: description,
          image: image,
          stock: stock,
          brand: brand,
          price: price,
          user: user._id,
        });
        res.status(201).json({
          message: "Product Created Successfully",
        });

    }catch (error) {
        console.log(error)
        return res.status(500).json({ message: "Internal Server Error" });
    }
}
```

to show single product

```
const SingleProduct = async(req,res)=>{
    try{
        const {id} = req.params;
        if(!id){
            res.status(400).json({message:"Product Id not found"});
        }

        const {token} = req.headers;

        const userEmailFromToken = jwt.verify(token,"supersecret");
        if(userEmailFromToken.email){
            const product = await Product.findById(id);
```

```
            if(!product){
                res.status(400).json({message:"Product not found"});
            }

            res.status(200).json({message:"success",product});
        }

    }catch(error){
        console.log(error);
        res.status(500).json({message:"Internal server error"});
    }
}
```

update the product

```
const productUpdate = async (req, res) => {
    const { id } = req.params;
    const { token } = req.headers;
    const body = req.body.productData;
    const name = body.name;
    const description = body.description;
    const image = body.image;
    const price = body.price;
    const brand = body.brand;
    const stock = body.stock;
    const userEmail = jwt.verify(token, "supersecret");
    try {
      if (userEmail.email) {
        const updatedProduct = await Product.findByIdAndUpdate(id, {
          name,
          description,
          image,
          price,
```

```
        brand,
        stock,
      });
      res.status(200).json({ message: "Product Updated Succesfully" });
    }
  } catch (error) {
    res.status(400).json({
      message: "Internal Server Error Occured While Updating Product",
    });
  }
}
```

delete product

```
const deleteProduct =  async(req,res)=>{
 try{
  const {id} = req.params;
  if(!id){
    res.status(400).json({message:" Product ID not found"});
  }
  const deleteProduct = await Product.findByIdAndDelete(id);

  if(!deleteProduct){
    res.status(404).json({message:"Product not found"});
  }

  res.status(200).json({
    message:"Product deleted successfully",
    product:deleteProduct
  });

 }catch(error){
  console.log(error);
```

```
    res.status(500).json({message:"Error deleting product",error});
  }
}

module.exports = {products,addProduct,SingleProduct,productUpdate,deletePro
```

index.js

```
const express = require("express");
const router = express.Router();
const UserRoutes = require("./user");
const ProductRoutes = require("./product");


router.use("/user",UserRoutes);
router.use("/product",ProductRoutes);


module.exports = router;
```

Cart

cartSchema.js

```
const mongoose = require('mongoose');

const cartSchema = new mongoose.Schema({
   products:[
      {
         type:mongoose.Schema.ObjectId,
         ref:"Product"
```

```
        }
    ],
    total:{
        type:Number
    }
})

const Cart = mongoose.model('Cart',cartSchema);


module.exports = {Cart};
```

routes → cart.js

```
const express = require("express");
const router = express.Router();
const {cart, addCart, deleteCart} = require('../controllers/cart.controller')

//task-8 → create cart route
router.get('/carts',cart)

// taks-9→ add product in cart
router.post("/add",addCart );

//task-10→ delete product from cart
router.delete("/product/delete",deleteCart);


module.exports = router;
```

controllers → cart.controller.js

cart

```
const {User} = require('../models/User');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const {Cart} = require("../models/Cart");
const { Product } = require('../models/Product');

const cart = async(req,res)=>{
  const {token} = req.headers;
  const decodedtoken = jwt.verify(token,"supersecret");
  const user = await User.findOne({email:decodedtoken.email}).populate({
    path:'cart',
    populate:{
      path:'products',
      model:'Product'
    }
  });
  if(!user){
   return res.status(400).json({message:"User not found"});
  }

  res.status(200).json( {cart:user.cart});
}
```

addcart

```
const addCart = async (req, res) => {
  const body = req.body;

  const productsArray = body.products;
  let totalPrice = 0;

  try {
```

```
  for (const item of productsArray) {
    const product = await Product.findById(item);
    if (product) {
      totalPrice += product.price;
    }
  }

  const { token } = req.headers;
  const decodedToken = jwt.verify(token, "supersecret");
  const user = await User.findOne({ email: decodedToken.email });

  if (!user) {
    return res.status(404).json({ message: "User Not Found" });
  }

  let cart;
  if (user.cart) {
    cart = await Cart.findById(user.cart).populate("products");
    const existingProductIds = cart.products.map((product) =>
      product._id.toString()
    );

    productsArray.forEach(async (productId) => {
      if (!existingProductIds.includes(productId)) {
        cart.products.push(productId);
        const product = await Product.findById(productId);
        totalPrice += product.price;
      }
    });

    cart.total = totalPrice;
    await cart.save();
  } else {
    cart = new Cart({
      products: productsArray,
      total: totalPrice,
```

```
      });

      await cart.save();
      user.cart = cart._id;
      await user.save();
    }

    res.status(201).json({
      message: "Cart Updated Successfully",
      cart: cart,
    });
  } catch (error) {
    res.status(500).json({ message: "Error Adding to Cart", error });
  }
}
```

delete

```
const deleteCart = async (req, res) => {
  const { productID } = req.body;
  const { token } = req.headers;

  try {
    const decodedToken = jwt.verify(token, "supersecret");
    const user = await User.findOne({ email: decodedToken.email }).populate("car

    if (!user) {
      return res.status(404).json({ message: "User Not Found" });
    }

    const cart = await Cart.findById(user.cart).populate("products");

    if (!cart) {
      return res.status(404).json({ message: "Cart Not Found" });
```

```
    }

    const productIndex = cart.products.findIndex(
      (product) => product._id.toString() === productID
    );

    if (productIndex === -1) {
      return res.status(404).json({ message: "Product Not Found in Cart" });
    }

    cart.products.splice(productIndex, 1);
    cart.total = cart.products.reduce(
      (total, product) => total + product.price,
      0
    );

    await cart.save();

    res.status(200).json({
      message: "Product Removed from Cart Successfully",
      cart: cart,
    });
  } catch (error) {
    res
      .status(500)
      .json({ message: "Error Removing Product from Cart", error });
  }
}

module.exports = {cart, addCart, deleteCart};
```

index.js

```
const express = require("express");
const router = express.Router();
const UserRoutes = require("./user");
const ProductRoutes = require("./product");
const CartRoutes = require('./cart')


router.use("/user",UserRoutes);
router.use("/product",ProductRoutes);
router.use('/cart',CartRoutes)


module.exports = router;
```